INSTITUT UNIVERSITAIRE SAINT JEAN
SAINT JEAN INGÉNIEUR

CAHIER DE CONCEPTION

Conception et développement de Bootability Checker,

un outil d'analyse de la bootabilité d'un disque : cas de FreeBSD

Rédigé par :

- DZOUAKEU FESSI Emmanuella Cindy
- YAKAM TCHAMOU Rick Vadel

Examinateur :

M. NGUIMBUS, Enseignant de Systèmes d'Exploitation

Année académique :

2025 - 2026

SOMMAIRE

I. Introduction
II. Architecture du système
III. Conception détaillée (UML)
IV. Patron de conception (Design pattern)
V. Conclusion

I. INTRODUCTION

Dans le cadre du développement de l'outil Bootability Checker, la phase de conception représente une étape fondamentale visant à traduire les besoins fonctionnels identifiés lors de l'analyse en une solution technique structurée et implémentable. Cette phase permet de définir de manière précise l'architecture du système, les interactions entre ses différents composants ainsi que les choix technologiques retenus.

Le présent cahier de conception a pour objectif de décrire l'organisation globale de l'application, les mécanismes internes utilisés pour analyser la bootabilité des disques sous FreeBSD, ainsi que les structures de données manipulées. Il constitue ainsi une référence essentielle pour garantir la cohérence, la robustesse et la maintenabilité de la solution développée.

II. ARCHITECTURE DU SYSTÈME

2.1 Vue globale

L'outil Bootability Checker est conçu selon une architecture modulaire reposant sur une interface en ligne de commande (CLI). Cette organisation permet une séparation claire des responsabilités entre les différents composants du système, facilitant ainsi la maintenance, l'évolution et la lisibilité du code.

L'architecture s'articule autour de plusieurs couches fonctionnelles. L'interface utilisateur assure l'interaction avec l'utilisateur, notamment via la saisie d'arguments ou l'utilisation d'un mode interactif. Elle transmet les requêtes au module de contrôle, qui joue un rôle central dans l'orchestration des traitements en coordonnant les différents modules du système.

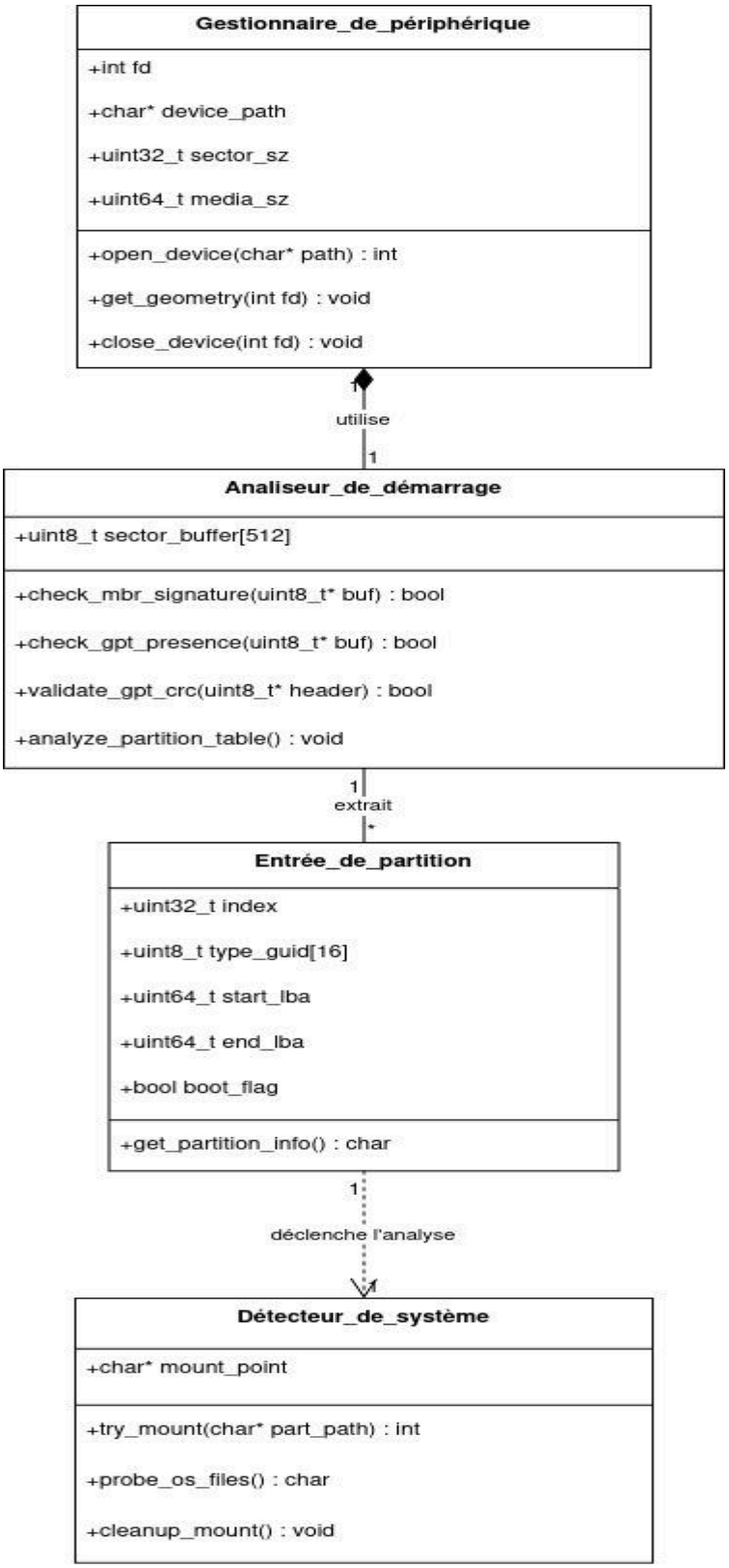
Le module d'analyse disque intervient ensuite pour effectuer une lecture bas niveau des données à partir des périphériques bloc. Cette lecture est réalisée en mode sécurisé, en accès strictement en lecture seule, afin de préserver l'intégrité des données. Les informations collectées sont ensuite exploitées par le module de détection, chargé d'identifier le type de partition utilisé (MBR ou GPT) ainsi que les structures de boot associées.

Enfin, le module de reporting est responsable de la synthèse des résultats. Il produit un rapport clair et structuré indiquant l'état de bootabilité du disque ainsi que le mode de démarrage détecté (BIOS ou UEFI), permettant à l'utilisateur d'interpréter rapidement les résultats de l'analyse.

III. CONCEPTION DÉTAILLÉE (UML)

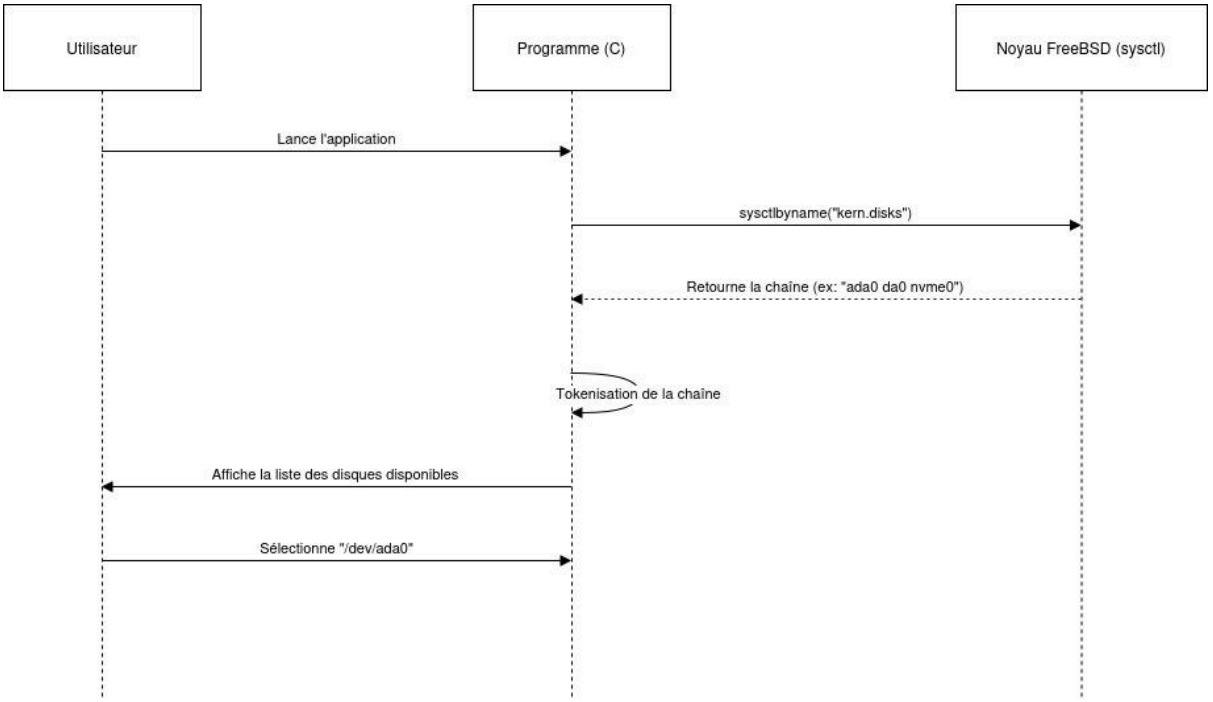
3.1 Diagramme de Classe

L'organisation du code est structurée autour de quatre entités principales pour assurer une séparation claire des responsabilités (Ségrégation des interfaces).



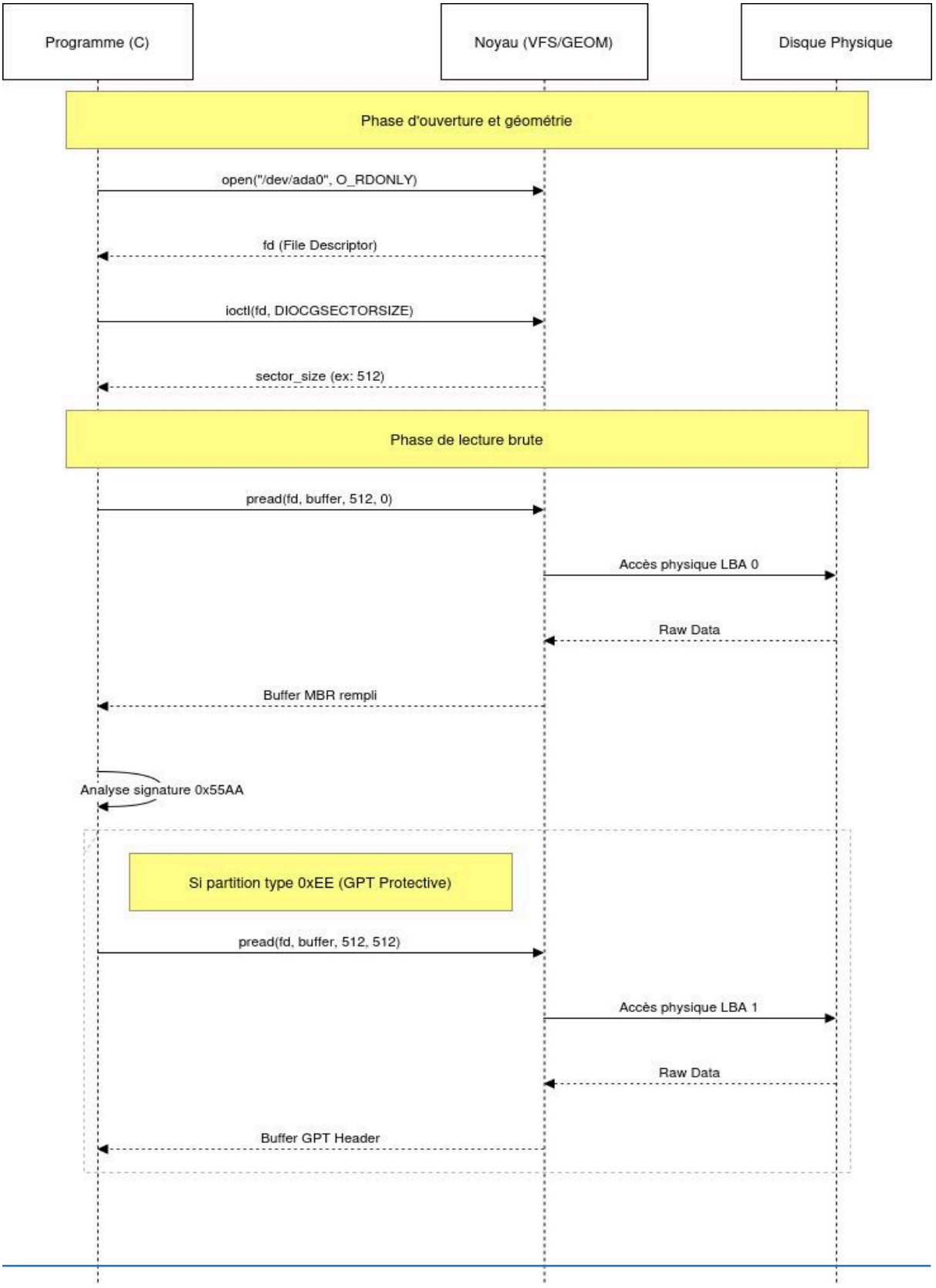
3.2 Diagramme de Séquence : Initialisation et Découverte

Ce diagramme montre comment le programme interroge le noyau pour lister les disques disponibles avant que l'utilisateur ne fasse son choix.



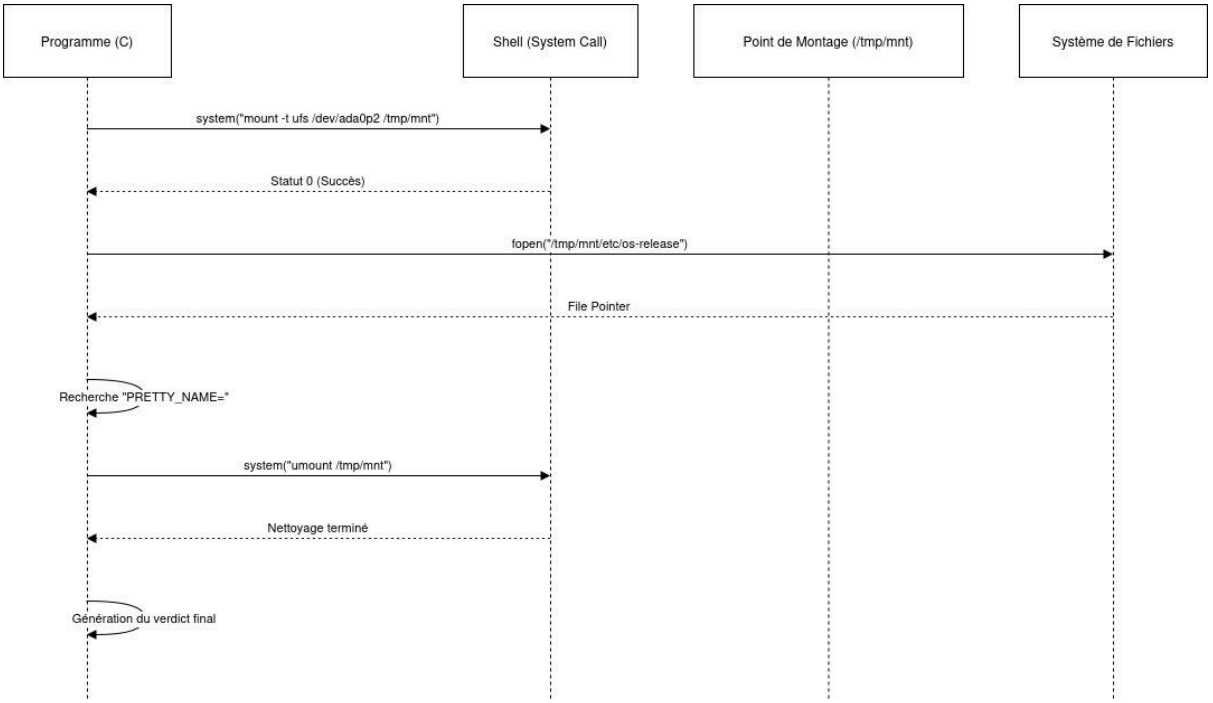
3.3 Diagramme de Séquence : Analyse de la Structure (LBA 0 & 1)

C'est le cœur technique. Il illustre l'usage de `ioctl` pour la géométrie et de `pread` pour la lecture brute afin de décider entre MBR et GPT.



3.4 Diagramme de Séquence : Identification de l'OS (Heuristique)

Ce diagramme détaille la stratégie "agressive" où le programme tente de monter la partition pour lire les fichiers système.



IV. PATRON DE CONCEPTION (DESIGN PATTERN)

Le projet implémente le patron **Stratégie (Strategy)**.

4.1 Justification du choix

Compte tenu de la coexistence des standards BIOS (Legacy) et UEFI (Moderne), le programme doit adapter son comportement dynamiquement. Le patron Stratégie permet d'isoler la logique de vérification MBR de celle du GPT, facilitant ainsi l'ajout futur de schémas plus rares (APM, BSD labels).

4.2 Modélisation du Pattern

- **Contexte :** Le moteur de scan.
- **Stratégie A (MBR) :** Analyse du code machine au LBA 0 et des drapeaux de boot (0x80).
- **Stratégie B (GPT) :** Analyse des GUID et vérification de l'intégrité par CRC32.

V. CONCLUSION

Le présent cahier de conception a permis de définir une architecture claire, modulaire et robuste pour le développement de l'outil Bootability Checker. Les choix techniques effectués, notamment en matière de lecture bas niveau et de séparation des responsabilités, garantissent une solution fiable, performante et facilement maintenable.

En s'appuyant sur une organisation structurée et des mécanismes d'analyse précis, cette conception constitue une base solide pour l'implémentation de l'outil et son évolution future. Elle ouvre également la voie à des améliorations potentielles, telles que l'intégration de nouvelles méthodes de détection ou l'extension à d'autres environnements systèmes.